Combined deck · 50 slides

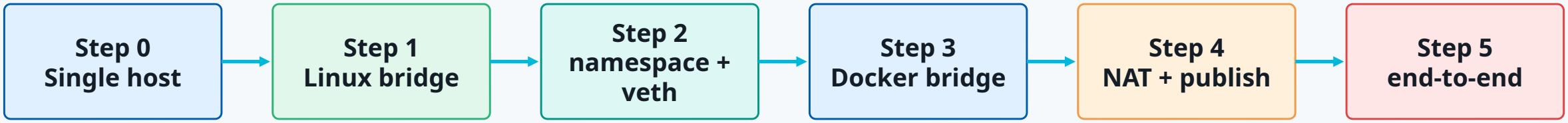
Introduction to Linux Networking for Containers

Single host foundations · bridges · namespaces · Docker bridge · NAT/port publishing

One practical lab: from single Linux host to Docker published port

Course storyline

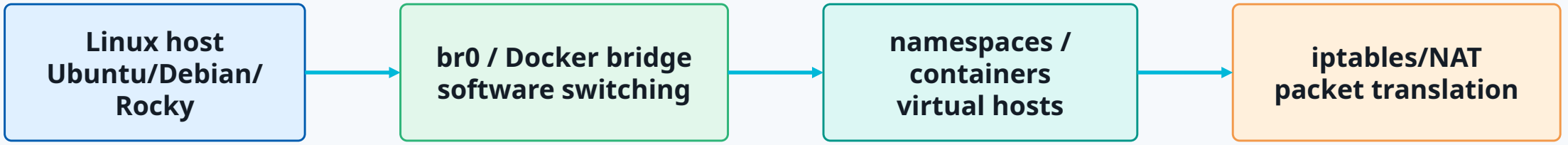
A single practical lab grows into a container networking mental model



One continuous demo: Linux host → br0 → namespaces → Docker bridge → published nginx port

Lab/demo topology

Everything runs on a single Linux VM or bare-metal host



Prerequisites

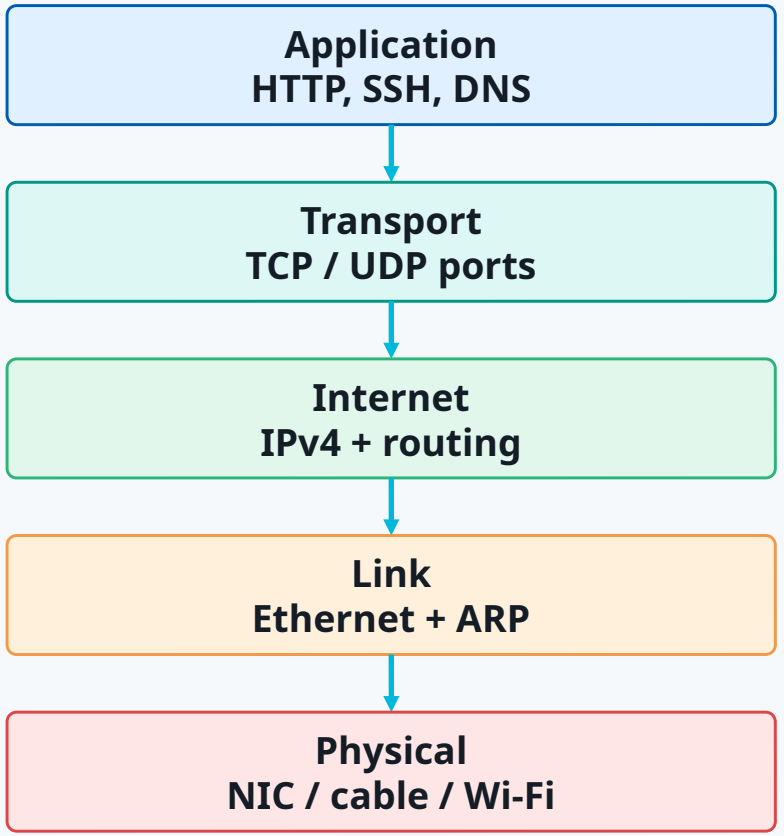
```
$ sudo apt install docker.io iproute2 iputils-ping tcpdump conntrack nginx curl
$ sudo usermod -aG docker $USER
# log out and back in after adding docker group
```

Step 0

Single Host Configuration

Practical TCP/IP stack

Every later container packet still follows this model



Troubleshooting rule: prove each layer before moving down

Layer-to-command mapping

Use the right tool for the layer you are testing

Layer	What you check	Linux commands
Application	Service, URL, DNS name	curl, dig, getent, logs
Transport	TCP/UDP ports and sessions	ss, nc, **tcpdump**
Internet	IP address and route	ip addr, ip route, ping
Link	MAC, ARP/neighbor, state	ip link, ip neigh, ethtool
Physical	Carrier, driver, medium	ethtool, dmesg, cable

IPv4 ranges: public vs private

Private addresses need routing/NAT to reach the Internet in most labs

Range	Prefix	Typical use
10.0.0.0 – 10.255.255.255	10.0.0.0/8	Enterprise/internal networks
172.16.0.0 – 172.31.255.255	172.16.0.0/12	Internal/container networks
192.168.0.0 – 192.168.255.255	192.168.0.0/16	Home/small office/LAN labs
Public IPv4	Internet-routable space	External services

Modern routing uses CIDR/prefix length, not old classful thinkin

Read host addressing

Start by observing before changing configuration

Terminal

```
$ ip -br link
lo        UNKNOWN   00:00:00:00:00:00
ens160    UP        00:50:56:aa:bb:cc

$ ip -br addr
ens160    UP        192.168.10.25/24 fe80::250:56ff:feaa:bbcc/64
```

Key: address + prefix + interface state

Routing: what path will the kernel choose?

ip route get is the fastest routing sanity check

```
Terminal
$ ip route
default via 192.168.10.1 dev ens160
192.168.10.0/24 dev ens160 proto kernel scope link src 192.168.10.25

$ ip route get 8.8.8.8
8.8.8.8 via 192.168.10.1 dev ens160 src 192.168.10.25
```

Linux uses longest prefix match, just like routers

DNS and sockets are separate problems

Do not confuse routing, DNS and application binding

```
Terminal
$ getent hosts example.com
93.184.216.34 example.com

$ ss -ltnp
LISTEN 0.0.0.0:22       sshd
LISTEN 127.0.0.1:8080   python
```

If ping 8.8.8.8 works but names fail: suspect DNS

Step 0 demo checkpoint

Single-host baseline before virtual networking

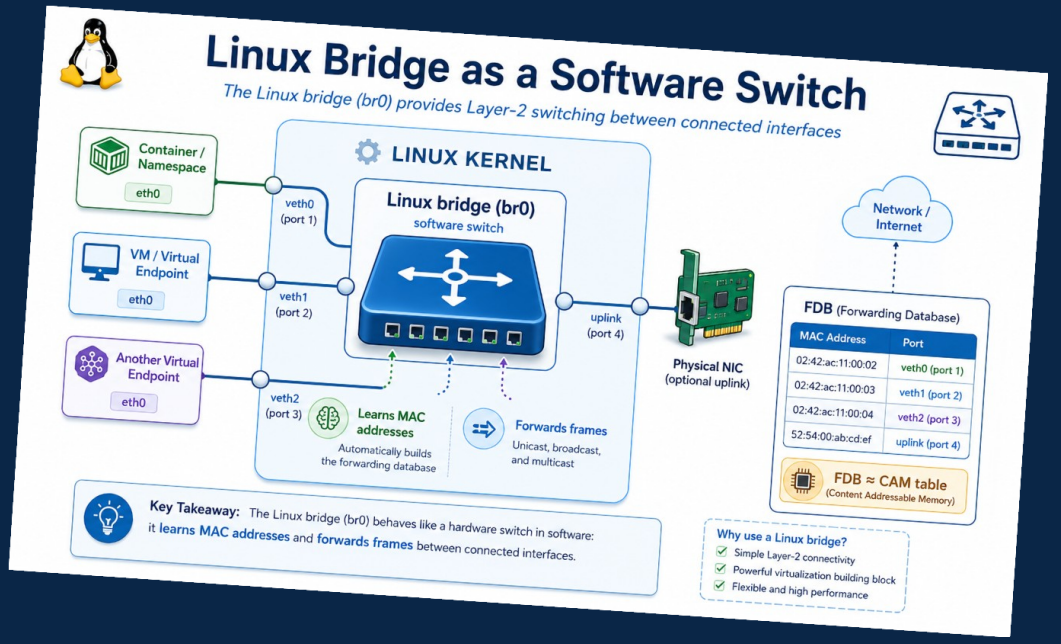
Demo commands

```
$ ip -br link
$ ip -br addr
$ ip route get 8.8.8.8
$ ip neigh
$ getent hosts example.com
$ ss -ltnp
$ tcpdump -ni any icmp
```

Expected outcome
You can identify interface, IP, route, DNS, listeners and packets

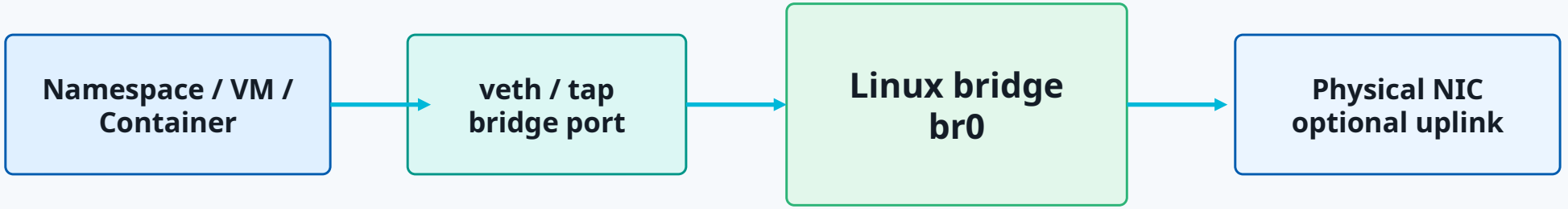
Step 1

Linux bridge as a software switch



Linux bridge mental model

A Linux bridge acts like a software Layer 2 switch



Cisco analogy: bridge FDB ≈ CAM table

Create br0

First lab step: create a Linux bridge

Terminal

```
$ sudo ip link add br0 type bridge
$ sudo ip link set br0 up
$ ip -br link show br0
br0       UP       8e:5a:0d:40:11:01
```

br0 is now a software switch with no ports attached yet

Inspect bridge ports and FDB

Linux bridge forwarding is MAC-based

Terminal

```
$ bridge link
# shows interfaces enslaved to a bridge

$ bridge fdb show br br0
52:54:00:aa:10:11 dev veth1 master br0
52:54:00:aa:10:12 dev veth2 master br0
```

FDB answers: which MAC lives behind which bridge port?

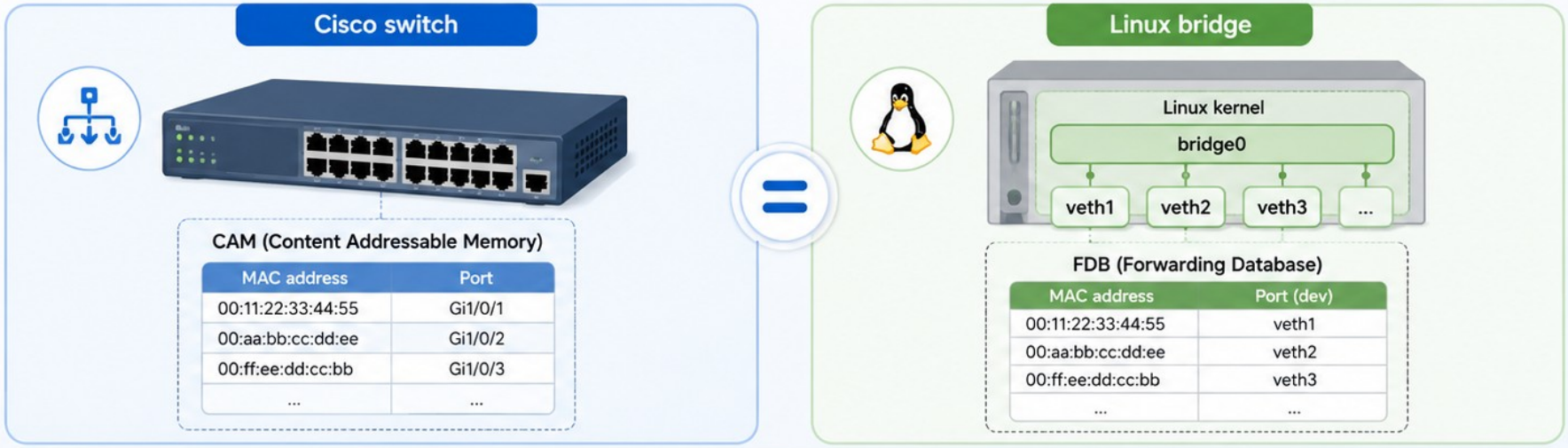
Cisco CAM vs Linux FDB

Same switching logic, different command surface

Cisco switch	Linux bridge	Purpose
show mac address-table	bridge fdb show	learned MAC addresses
interface Gi1/0/1	dev veth1	port where MAC is learned
VLAN-aware switch	bridge vlan	VLAN filtering/port membership
CAM table troubleshooting	FDB troubleshooting	find where frames go

Cisco CAM vs Linux FDB

Same switching logic, different command surface



Cisco switch	Linux bridge	Purpose
show mac address-table	bridge fdb show	learned MAC addresses
interface Gi1/0/1	dev veth1	port where MAC is learned
VLAN-aware switch	bridge vlan	VLAN filtering/port membership
CAM table troubleshooting	FDB troubleshooting	find where frames go

Same Layer-2 forwarding logic. Different commands.

CAM table in Cisco ≈ FDB in Linux bridge

Step 1 demo checkpoint

You have created and inspected a virtual switch

Demo commands

```
$ ip link add br0 type bridge
$ ip link set br0 up
$ ip link show type bridge
$ bridge link
$ bridge fdb show br br0
```

Expected outcome
You can explain br0 as a software switch and FDB as a CAM-like table

Step 2

Network namespace + veth + bridge

Network namespace mental model

A namespace is an isolated network stack

Default host namespace
ens160, routes, sockets

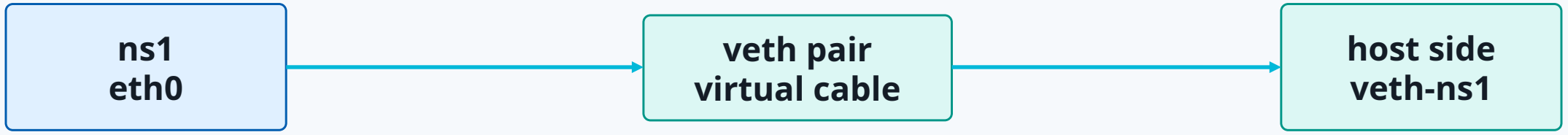
ns1
eth0, routes, sockets

ns2
eth0, routes, sockets

Cisco-friendly analogy: a namespace is like a lightweight isolated host/VRF context

veth pair mental model

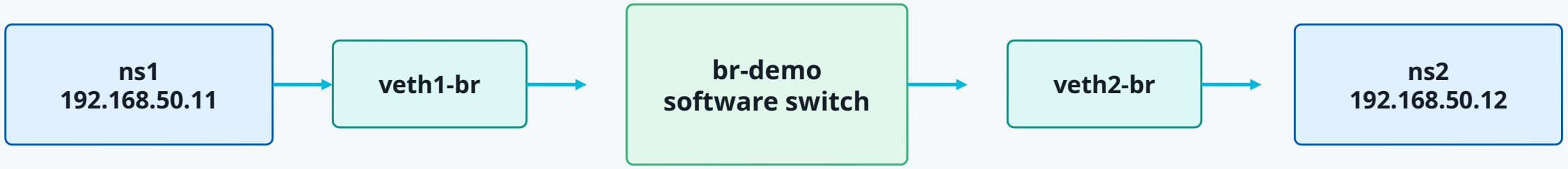
A veth pair is a virtual Ethernet cable



Anything entering one side exits the other side

Namespace + veth + bridge topology

This is the manual version of a basic container network



Build the namespace lab

Create namespaces, bridge and veth pairs

Terminal

```
$ sudo ip netns add ns1
$ sudo ip netns add ns2
$ sudo ip link add br-demo type bridge
$ sudo ip link set br-demo up
$ sudo ip link add veth1-br type veth peer name eth0 netns ns1
$ sudo ip link add veth2-br type veth peer name eth0 netns ns2
```

Attach and address

Host-side veth interfaces become bridge ports

Terminal

```
$ sudo ip link set veth1-br master br-demo
$ sudo ip link set veth2-br master br-demo
$ sudo ip link set veth1-br up
$ sudo ip link set veth2-br up
$ sudo ip -n ns1 addr add 192.168.50.11/24 dev eth0
$ sudo ip -n ns2 addr add 192.168.50.12/24 dev eth0
$ sudo ip -n ns1 link set eth0 up
$ sudo ip -n ns2 link set eth0 up
```

Test and inspect

Ping across the bridge and inspect the FDB

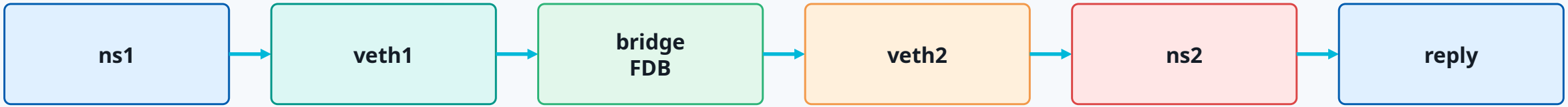
```
Terminal
$ sudo ip netns exec ns1 ping -c 2 192.168.50.12
64 bytes from 192.168.50.12: icmp_seq=1 ttl=64 time=0.09 ms

$ bridge fdb show br br-demo
52:54:00:aa:10:11 dev veth1-br master br-demo
52:54:00:aa:10:12 dev veth2-br master br-demo

$ sudo tcpdump -ni br-demo icmp
```

Step 2 checkpoint

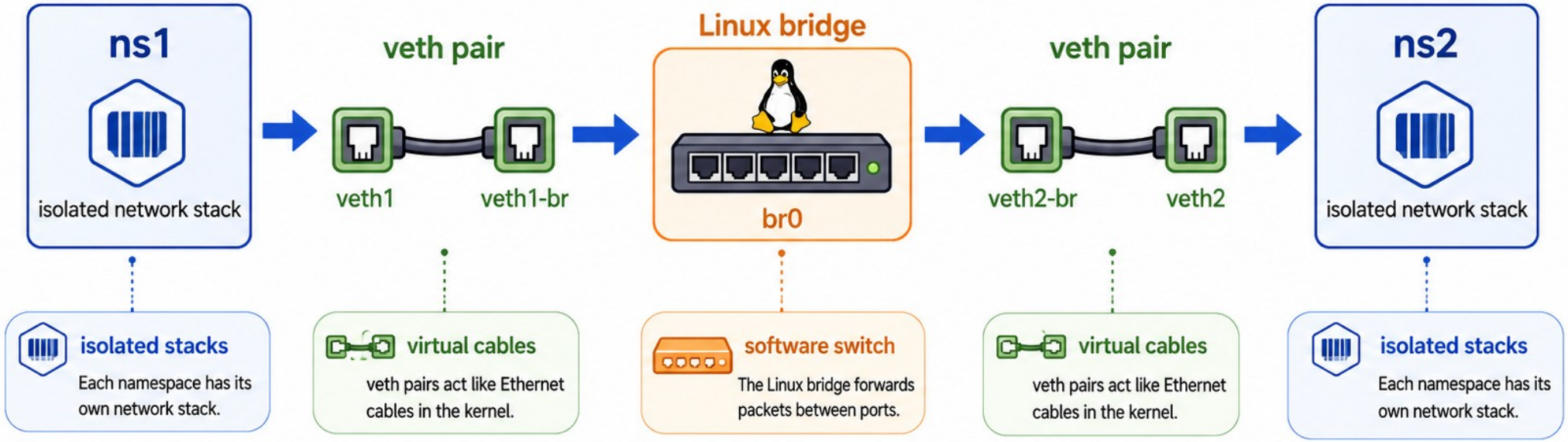
The packet path is now visible



Manual container-like network: isolated stacks + virtual cables + software switch

Step 2 checkpoint

The packet path is now visible



We manually built a container-like network.

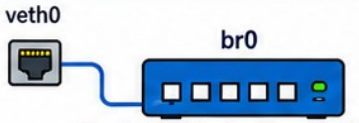
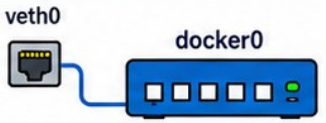
Docker will later automate a very similar structure.

Step 3

Docker bridge networking

Docker maps to the primitives

Docker automates the topology you built manually

Manual lab	Docker equivalent
1 `ip netns add ns1` 	 container network namespace
2 `ip link add ... type veth` 	 Docker creates veth pair
3 `ip link set veth master br0` 	 Docker attaches host veth to bridge
4 `ip addr add ...` 	 Docker IPAM assigns container IP
5 `bridge fdb show`	bridge learns container MACs

FDB (MAC table)

MAC address	Port	Age
02:42:0a:00:00:02	veth0	10s
02:42:0a:00:00:03	veth1	12s
...		

FDB (MAC table)

MAC address	Port	Age
02:42:0a:00:00:02	veth0	10s
02:42:0a:00:00:03	veth1	12s
...		

Docker is not magic —
it automates the same Linux networking primitives we built manually.

Inspect Docker default bridge

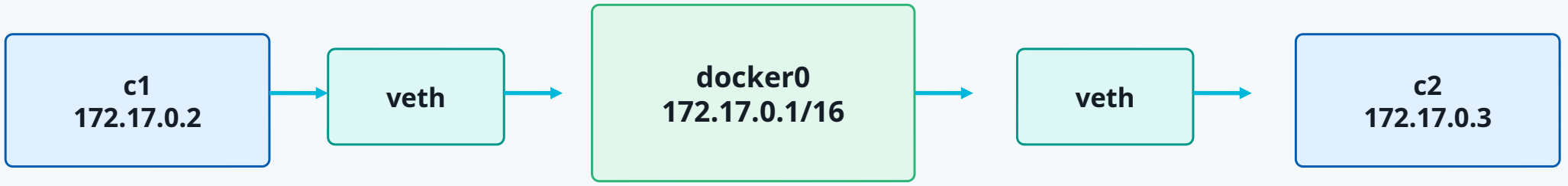
Start by looking at Docker network state

```
Terminal
$ docker network ls
NETWORK ID     NAME      DRIVER    SCOPE
6b7c5e7f9a01   bridge    bridge    local
b58d781f1234   host      host      local
0f2d9ef77821   none      null      local

$ docker network inspect bridge
```

Default bridge topology

Containers on the same default bridge can communicate by IP



Run two containers on default bridge

Default bridge works by IP; name resolution is limited

```
Terminal
$ docker run -dit --name c1 alpine sh
$ docker run -dit --name c2 alpine sh
$ docker inspect -f "{{range.NetworkSettings.Networks}}{{.IPAddress}}{{end}}" c2
172.17.0.3
$ docker exec c1 ping -c 2 172.17.0.3   # works
$ docker exec c1 ping -c 1 c2           # usually fails on default bridge
```

Create a user-defined bridge

Preferred pattern for multi-container apps

Terminal

```
$ docker network create app-net
$ docker run -dit --name web --network app-net nginx:alpine
$ docker run -dit --name client --network app-net alpine sh
$ docker network inspect app-net
```

User-defined bridge gives per-app isolation and built-in container-name DNS

DNS resolution between containers

Container names become practical service-discovery names

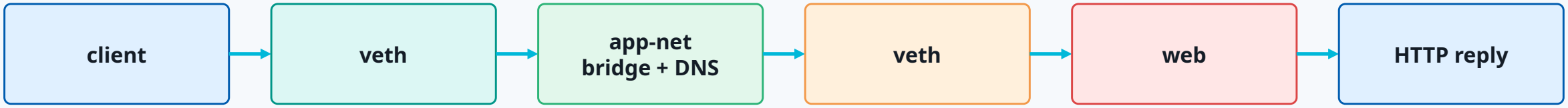
Terminal

```
$ docker exec client cat /etc/resolv.conf
nameserver 127.0.0.11

$ docker exec client getent hosts web
172.18.0.2      web
$ docker exec client wget -qO- http://web | head
<!DOCTYPE html>
```

Step 3 checkpoint

Docker bridge is now understandable



The same namespace + veth + bridge model is now automated by Docker

Step 4

Docker NAT / port publishing

Step 4: Docker NAT / Port Publishing

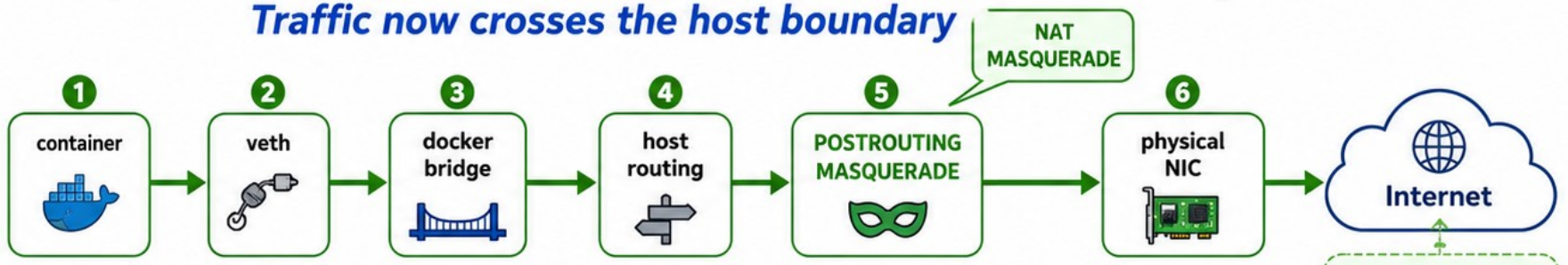
Traffic now crosses the host boundary

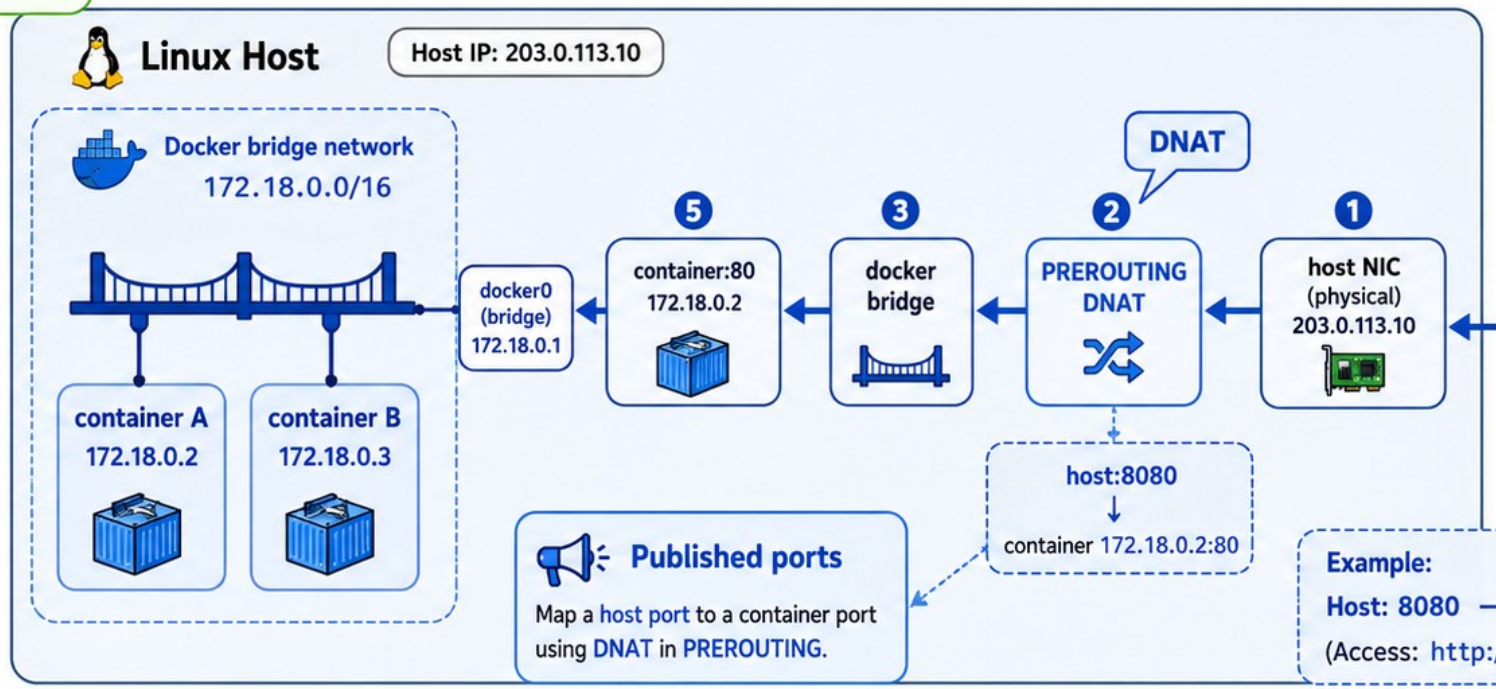
OUTBOUND (container → Internet)

Containers can reach the Internet.

Outbound traffic uses NAT/MASQUERADE.

The outside world sees the host IP.





Key Concepts

NAT / MASQUERADE
Used for outbound traffic. Source is rewritten to host IP.

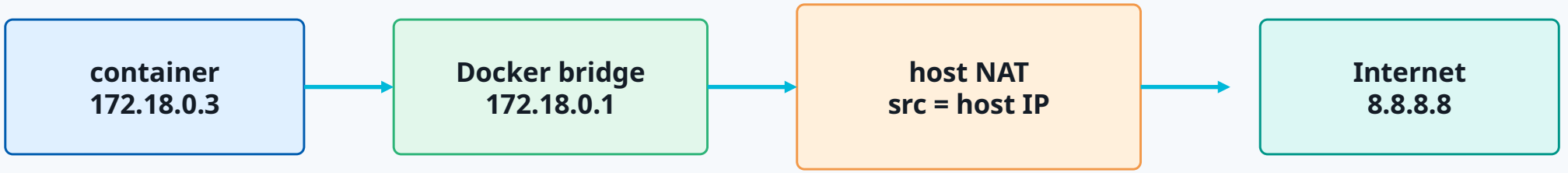
DNAT
Used for incoming traffic. Destination is rewritten to container IP:port.

Published ports
Expose container services to the outside world.

Until now, we mainly looked at communication between containers on the same bridge network.
Now, Docker networking includes traffic leaving the host (NAT/MASQUERADE) and traffic entering published container services (DNAT).

Why Docker needs NAT

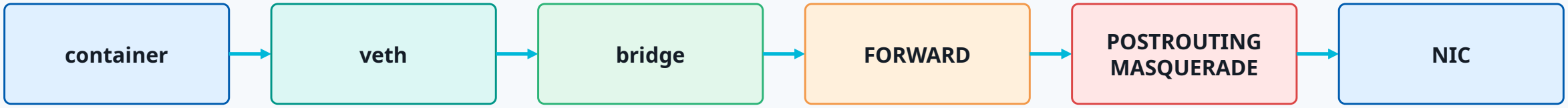
Containers use private addressing; external networks usually do not route back



Outbound: POSTROUTING MASQUERADE changes source from container IP to host IP

Outbound NAT path

Container reaching the Internet



The outside world usually sees the host IP, not the container IP

Inspect NAT and conntrack

NAT is both rules and state

```
Terminal
$ sudo iptables-save | sed -n "/\*nat/,/COMMIT/p"
-A POSTROUTING -s 172.18.0.0/16 ! -o br-xxxx -j MASQUERADE

$ sudo conntrack -L | grep 172.18.0.3
tcp 6 src=172.18.0.3 dst=93.184.216.34 sport=51234 dport=443 ...
```

conntrack remembers translations so return traffic reaches the correct container

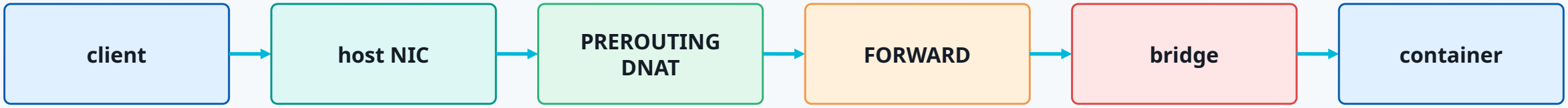
Publish a container port

External client reaches host port; Docker DNAT forwards to container

```
Terminal
$ docker run -d --name web --network app-net -p 8080:80 nginx:alpine
$ docker ps --format "table {{.Names}}\t{{.Ports}}"
NAMES   PORTS
web     0.0.0.0:8080->80/tcp
$ curl -I http://127.0.0.1:8080
HTTP/1.1 200 OK
```

Inbound published-port path

DNAT changes destination from host:8080 to container:80



Important: published container traffic typically uses FORWARD, not only INPUT

Inspect DNAT rules

Docker-managed chains contain the translation

```
Terminal
$ sudo iptables-save | grep -E "8080|DNAT|DOCKER"
-A PREROUTING -m addrtype --dst-type LOCAL -j DOCKER
-A DOCKER ! -i br-xxxx -p tcp --dport 8080 \
    -j DNAT --to-destination 172.18.0.2:80
```

Destination changes from host:8080 to container:80 before forwarding

INPUT vs FORWARD

The most common firewall confusion

Traffic	Destination after routing/NAT	Important chain/path
SSH to host	Linux host itself	INPUT
host:8080 → container:80	container behind bridge	PREROUTING DNAT + FORWARD
container → Internet	external destination	FORWARD + POSTROUTING NAT
host process → Internet	external destination	OUTPUT + POSTROUTING

Correct rule in the wrong chain is still wrong

Packet capture proves NAT

Capture before and after the translation point

```
Terminal
# before outbound NAT: on bridge
$ sudo tcpdump -ni br-xxxx host 8.8.8.8
IP 172.18.0.3 > 8.8.8.8: ICMP echo request

# after outbound NAT: on physical NIC
$ sudo tcpdump -ni ens160 host 8.8.8.8
IP 10.0.0.20 > 8.8.8.8: ICMP echo request
```

Step 4 checkpoint

You can now explain Docker egress and published ports

Outbound container → Internet

SNAT/MASQ POSTROUTING

Inbound host:8080 → container:80

DNAT PREROUTING + FORWARD

Docker NAT = DNAT + MASQUERADE + FORWARD + conntrack

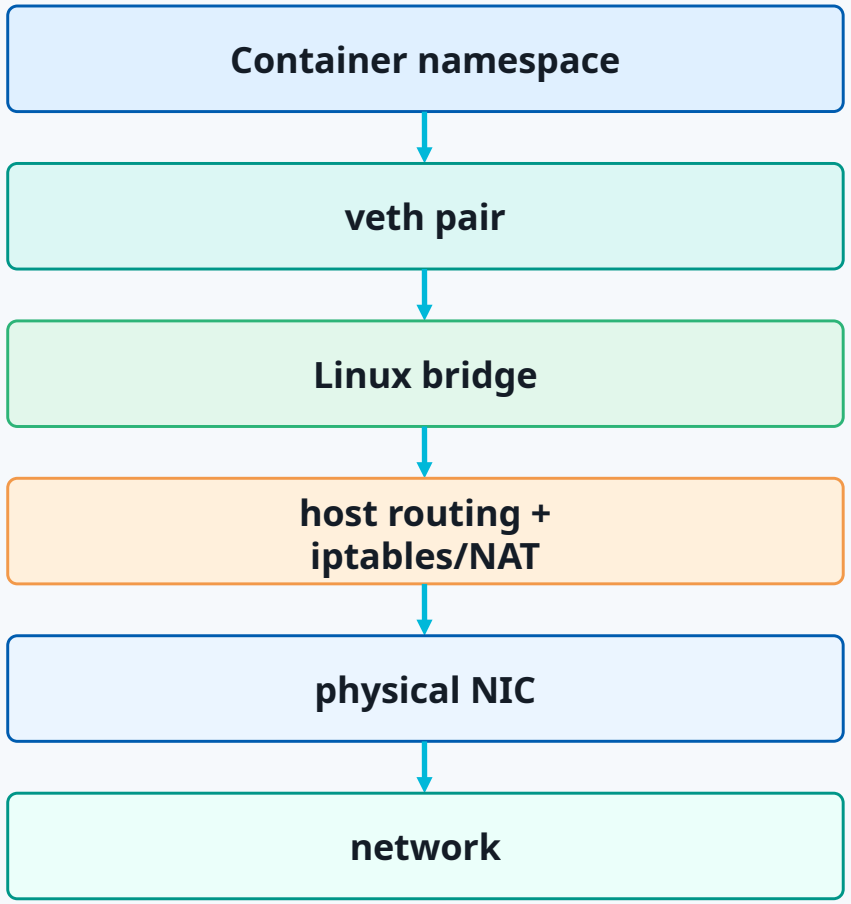
Step 5

End-to-end mental model and practical lab

The main mental model

This is the diagram you should be able to draw from memory

Every troubleshooting question becomes: Where is the packet now, and where should it go next?



Each layer is both a forwarding point and an inspection point.

End-to-end troubleshooting decision tree

Classify the failure before changing the configuration

No at step 1: application issue
No at step 2: namespace/container network issue
No at step 3: bridge/veth issue
No at step 4: Docker DNS/network attachment issue
No at step 5: firewall/NAT/routing issue

Practical lab: build and verify

One compact demo that touches all course concepts

Lab script · Part 1

```
# 1) host baseline
ip -br addr && ip route get 8.8.8.8
# 2) create app network and containers
docker network create app-net
docker run -d --name web --network app-net -p 8080:80 nginx:alpine
docker run -dit --name client --network app-net alpine sh
# 3) verify service discovery and HTTP
docker exec client getent hosts web
docker exec client wget -qO- http://web | head
```

Practical lab: inspect packet path

Show Docker from Linux host perspective

Lab script · Part 2

```
# Docker view
docker network inspect app-net
docker ps
# Linux bridge view
ip link show type bridge
bridge link
bridge fdb show
# NAT/firewall view
iptables-save | grep -E "8080|DNAT|MASQUERADE|DOCKER"
conntrack -L | grep 172.18
# packet evidence
tcpdump -ni any tcp port 8080
```

Cleanup and reset

Keep labs reproducible

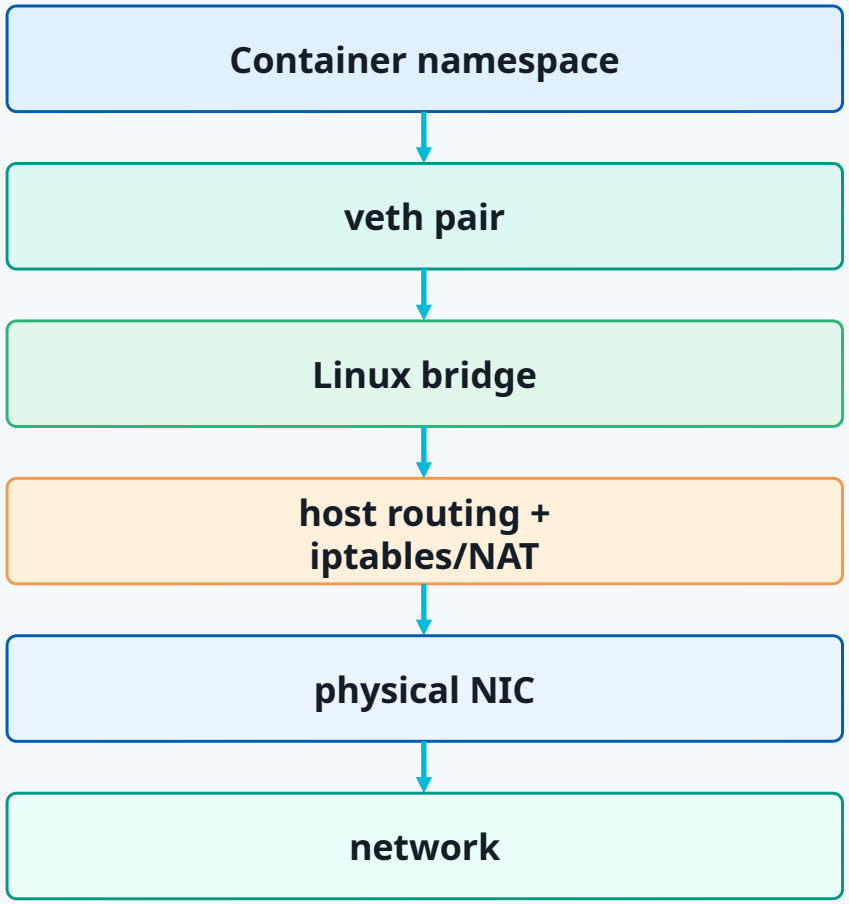
Cleanup commands

```
$ docker rm -f web client c1 c2
$ docker network rm app-net
$ sudo ip netns del ns1 2>/dev/null || true
$ sudo ip netns del ns2 2>/dev/null || true
$ sudo ip link del br-demo 2>/dev/null || true
$ sudo ip link del br0 2>/dev/null || true
$ docker network ls
$ ip -br link
```

Clean state = easier troubleshooting and repeatable demos

Final summary

The whole course in one sentence



Containers are not magic.
They are Linux networking primitives automated by Docker.

Single host TCP/IP gives you the baseline.
Bridge gives you software switching.
Namespace gives you isolation.
veth gives you the cable.
Docker gives you automation.
iptables/NAT gives you outside connectivity.

Follow the packet — the model tells you where to look.